

Introducción a la programación.

M. en C. Saul Villegas.

Universidad Autónoma de la Ciudad de México.

21 de abril de 2026

Al finalizar este bloque, el estudiante será capaz de:

- Comprender qué es una función.
- Crear funciones propias.
- Diferenciar variables locales y globales.
- Comprender el paso de parámetros.
- Utilizar punteros para modificar variables.
- Aplicar recursividad.
- Separar programas en módulos.

¿Por qué existen las funciones?

Cuando un programa crece

- Repetir código genera errores.
- Cuesta mantenerlo.
- Es difícil entenderlo.

Solución

Usar funciones permite dividir problemas grandes en partes pequeñas, claras y reutilizables.

¿Qué es una función?

- Es un bloque de instrucciones.
- Realiza una tarea específica.
- Puede recibir datos.
- Puede regresar un resultado.

Ejemplo en C

```
int sumar(int a, int b){  
    return a + b;  
}
```

Ventajas de usar funciones

- Reutilización de código.
- Programas más limpios.
- Más fáciles de corregir.
- Facilitan el trabajo en equipo.
- Reducen el tiempo de desarrollo.

Anatomía de una función

Estructura general

```
tipo nombre(parametros){  
    instrucciones;  
    return valor;  
}
```

Ejemplo

```
float promedio(float a, float b){  
    return (a + b) / 2;  
}
```

Partes

- Tipo de retorno.
- Nombre.
- Parámetros.
- Cuerpo.
- Retorno.

Prototipo, definición y llamada

Prototipo

```
int sumar(int a, int b);
```

Definición

```
int sumar(int a, int b){  
    return a + b;  
}
```

Llamada

```
r = sumar(3, 5);
```

Primero se declara, luego se define y finalmente se utiliza.

Paso de parámetros y punteros

1. Paso por valor

```
void cambiar(int x){  
    x = 100;  
}  
  
int a = 5; cambiar(a);
```

Resultado: a = 5. No cambia.

2. Para modificar la original

```
int a = 5, *p = &a;
```

- a: valor.
- &a: dirección.
- p: guarda dirección.
- *p: accese al contenido.

3. Paso tipo referencia

```
void cambiar(int *x){  
    *x = 100;  
}  
  
int main(){  
    int a = 5; cambiar(&a);  
}
```

Ahora: a = 100.

Variables locales y globales

Locales

Solo existen dentro de la función.

```
void f(){  
    int x = 5;  
}
```

Globales

Se declaran fuera de las funciones.

```
int contador = 0;
```

Pueden usarse en varias funciones.

Usar preferentemente variables locales

- Reducen efectos secundarios.
- Facilitan pruebas y depuración.
- Hacen más claro el comportamiento de cada función.

Evitar exceso de globales

- Generan errores ocultos.
- Dificultan el mantenimiento.
- Complican el trabajo en equipo.

Ejemplo: factorial

```
int fact(int n){  
    if(n <= 1) return 1;  
    return n * fact(n - 1);  
}
```

Idea clave

Una función puede llamarse a sí misma para resolver un problema en pasos más pequeños.

Ejemplo de evaluación

$\text{fact}(4) = 4 * 3 * 2 * 1$

¿Cuándo usar recursividad?

La recursividad es útil en problemas como:

- Factorial.
- Fibonacci.
- Árboles.
- Búsquedas.
- Algoritmos matemáticos.

Criterio práctico

Se recomienda cuando un problema se descompone naturalmente en subproblemas similares.

Separar el programa en archivos

- `main.c`
- `operaciones.c`
- `operaciones.h`

Ventajas

- Orden.
- Mantenimiento.
- Escalabilidad.
- Trabajo colaborativo.

Ejemplo modular

operaciones.h

```
int sumar(int, int);
```

operaciones.c

```
int sumar(int a, int b){  
    return a + b;  
}
```

main.c

```
printf("%d", sumar(2, 3));
```

Situaciones que debemos evitar

- No declarar el prototipo.
- Usar tipos incompatibles.
- No retornar un valor cuando corresponde.
- Confundir *p con p.
- Intentar usar variables fuera de su alcance.

Crear un programa que:

- 1 Lea dos números.
- 2 Use una función de suma.
- 3 Use una función de multiplicación.
- 4 Use un puntero para intercambiar valores.
- 5 Muestre los resultados.

Meta

Integrar funciones, parámetros y punteros en un mismo ejercicio.

Las funciones permiten escribir programas profesionales

- Funciones.
- Parámetros.
- Punteros.
- Variables locales y globales.
- Recursividad.
- Modularidad.

Reflexiona y responde

¿Por qué un programa dividido en funciones es mejor que uno de 500 líneas seguidas?

Modularidad = claridad + reutilización + mantenimiento